MATLAB Scripts

Contents

GA_TRIM_velocity.m

```
%% ===== GA-integrated Trim vs Velocity (robust toolbox/parallel detect) =====
clear; clc; close all; rng(1);

% --- Pick model ---
[filename, pathname] = uigetfile('*.slx', 'Select your Simulink model file');
if isequal(filename, 0), error('User canceled file selection.'); end
modelName = erase(filename, '.slx');

% --- Velocity sweep (knots) ---
velocity_settings = 110:5:160;      % sweep
n = numel(velocity_settings);
target_h = 5000;                    % ft (fixed height)

% --- GA decision variables: elevator schedule coeffs (radians), x=[a b c]
% delta_e(V) = a + b*xi + c*xi^2, where xi = (V - Vc)/scale
deg = pi/180;
nvar = 3;
lb   = [-12*deg, -20*deg, -20*deg];
ub   = [ +4*deg,  +20*deg,  +20*deg];

% --- Objective weights ---
w_dx = 1.0;      % weight on trim residual norm
w_h  = 0.0;      % optional height penalty (kept 0; we fix height)

% --- GA meta settings (used by both paths) ---
popSize     = 50;
generations = 60;

% ====================== Robust toolbox / parallel checks ======================
hasGAFcn   = ~isempty(which('ga'));          % function present?
hasGADSVer = ~isempty(ver('gads'));          % toolbox installed?
hasOptOpts = ~isempty(which('optimoptions'));
useGADS    = hasGAFcn && hasGADSVer && hasOptOpts && license('test','GADS_Toolbox');

hasParpool = ~isempty(which('parpool'));
usePCT     = hasParpool && license('test','Distrib_Computing_Toolbox');

if usePCT
   try
      if isempty(gcp('nocreate')), parpool; end
      fprintf('Parallel enabled.\n');
   catch ME
      warning('Could not start parallel pool: %s. Continuing single-threaded.', ME.message);
      usePCT = false;
   end
end

% --- Objective (shared): minimize sum of trim residuals across velocities
obj = @(x) objective_velocity_schedule_cost(x, modelName, velocity_settings, target_h, w_dx, w_h);

% ========================= Path A: Built-in GA (if OK) ========================
if useGADS
   fprintf('Built-in GA detected (functions + license). Using Global Optimization Toolbox.\n');
   try
      opts = optimoptions('ga', ...
         'PopulationSize',       popSize, ...
         'MaxGenerations',       generations, ...
         'UseParallel',          usePCT, ...
         'Display',              'iter', ...
         'FunctionTolerance',    1e-6, ...
         'MaxStallGenerations',  20);

      tic;
      [xbest, bestF] = ga(obj, nvar, [], [], [], [], lb, ub, [], opts);
      tsec = toc;
      fprintf('\n== GA (toolbox) best f = %.6g | time = %.2fs | coeffs (deg) = [%.2f %.2f %.2f]\n', ...
         bestF, tsec, xbest*180/pi);
      BestHist = [];   % not collecting per-generation history in toolbox path

   catch ME
      warning('Falling back to custom GA because built-in GA failed: %s', ME.message);
      useGADS = false;   % force fallback below
   end
end
```

```
% ========================= Path B: Custom GA (fallback) =========================
if ~useGADS
    fprintf('Using custom GA (no GADS on path / failed to init).\n');

    % Custom GA hyperparameters
    eliteFrac = 0.1;            % top % copied each gen
    pcross    = 0.8;            % crossover probability
    pmut      = 0.2;            % mutation probability
    mutSigma  = [2 5 5]*deg;    % mutation std dev per gene (radians)
    usePar    = usePCT;         % parfor if available

    % Initialize population
    Pop = repmat(lb, popSize,1) + rand(popSize,nvar).*repmat((ub-lb),popSize,1);
    BestHist = nan(generations,1);

    % Main loop
    for g = 1:generations
        % Fitness evaluation
        F = nan(popSize,1);
        if usePar
            parfor i = 1:popSize
                F(i) = obj(Pop(i,:));
            end
        else
            for i = 1:popSize
                F(i) = obj(Pop(i,:));
            end
        end

        % Sort, record best
        [F, idx] = sort(F);
        Pop = Pop(idx,:);
        BestHist(g) = F(1);
        fprintf('Gen %3d: best f = %.6g | x(deg) = [%.2f %.2f %.2f]\n', g, F(1), Pop(1,:)*180/pi);

        % Elitism
        nelite = max(1, round(eliteFrac*popSize));
        NewPop = Pop(1:nelite,:);

        % Offspring
        while size(NewPop,1) < popSize
            p1 = tournament_pick(F,3); p2 = tournament_pick(F,3);
            parent1 = Pop(p1,:); parent2 = Pop(p2,:);

            % Blend crossover
            if rand < pcross
                alpha  = rand(1,nvar);
                child1 = alpha.*parent1 + (1-alpha).*parent2;
                child2 = alpha.*parent2 + (1-alpha).*parent1;
            else
                child1 = parent1; child2 = parent2;
            end

            % Mutation
            child1 = child1 + (randn(1,nvar).*mutSigma).*(rand(1,nvar) < pmut);
            child2 = child2 + (randn(1,nvar).*mutSigma).*(rand(1,nvar) < pmut);

            % Bounds
            child1 = min(max(child1,lb),ub);
            child2 = min(max(child2,lb),ub);

            NewPop = [NewPop; child1; child2]; %#ok<AGROW>
        end

        % Next generation
        Pop = NewPop(1:popSize,:);
    end

    % Final pick
    F = nan(popSize,1);
    for i = 1:popSize, F(i) = obj(Pop(i,:)); end
    [bestF, bidx] = min(F); xbest = Pop(bidx,:);
    fprintf('\n== GA (custom) best f = %.6g | coeffs (deg) = [%.2f %.2f %.2f]\n', bestF, xbest*180/pi);
end

% ================== Evaluate best schedule & produce your outputs ==============
[xi, ~, ~] = normalize_velocity(velocity_settings);
elev_sched = xbest(1) + xbest(2)*xi + xbest(3)*xi.^2;   % radians

% Preallocate outputs (match your original reporting)
alpha_array     = zeros(1, n);  % AoA (rad)
q_array         = zeros(1, n);  % pitch rate (rad/s)
elevator_array  = zeros(1, n);  % elevator (rad)
```

```
throttle_array  = zeros(1, n);  % required throttle (hp)
dx_norm         = zeros(1, n);  % ||dx||
dx_all          = zeros(5, n);  % state derivatives
summary_table   = [];

for i = 1:n
    V = velocity_settings(i);
    elev_i = elev_sched(i);                    % GA-fixed elevator at this V
    [x_trim, u_trim, y_trim, dx_trim] = run_trim_velocity(modelName, V, elev_i, target_h);
    alpha_array(i)    = x_trim(2);
    q_array(i)        = x_trim(4);
    elevator_array(i) = u_trim(1);
    throttle_array(i) = u_trim(2);
    dx_norm(i)        = norm(dx_trim);
    dx_all(:, i)      = dx_trim;

    summary_table = [summary_table; ...
        V, u_trim(2), rad2deg(x_trim(2)), rad2deg(x_trim(4)), rad2deg(u_trim(1)), norm(dx_trim)];
end

% Convert to degrees for plots
alpha_deg = rad2deg(alpha_array);
q_deg_s   = rad2deg(q_array);
elev_deg  = rad2deg(elevator_array);

% Optional convergence plot (custom GA only)
if ~isempty(BestHist)
    figure; plot(1:numel(BestHist), BestHist, 'LineWidth',2);
    xlabel('Generation'); ylabel('Best objective'); grid on; title('GA convergence (custom)');
end

%% Plot 1: Combined Throttle and AoA vs Velocity
figure;
yyaxis left
plot(velocity_settings, throttle_array, '-o', 'LineWidth', 2, 'Color', '#D95319');
ylabel('Required Throttle (hp)');
yyaxis right
plot(velocity_settings, alpha_deg, '-s', 'LineWidth', 2, 'Color', '#0072BD');
ylabel('Angle of Attack (deg)');
xlabel('Trim Velocity (knots)');
title('Required Throttle and AoA vs Velocity');
legend('Throttle', 'Angle of Attack', 'Location', 'best');
grid on;

%% Plot 2: Throttle vs Velocity
figure;
plot(velocity_settings, throttle_array, '-o', 'LineWidth', 2, 'Color', '#EDB120');
grid on;
xlabel('Trim Velocity (knots)');
ylabel('Required Throttle (hp)');
title('Throttle Requirement vs Velocity');
legend('Throttle vs Velocity', 'Location', 'best');

%% Best trim report
[~, bestIdx] = min(dx_norm);
fprintf('\n=== Best Trim Condition ===\n');
fprintf('Velocity           : %.1f knots\n', velocity_settings(bestIdx));
fprintf('Required Throttle  : %.2f hp\n', throttle_array(bestIdx));
fprintf('Angle of Attack    : %.2f deg\n', alpha_deg(bestIdx));
fprintf('Elevator Deflection: %.2f deg\n', elev_deg(bestIdx));
fprintf('Pitch Rate (q)     : %.2f deg/s\n', q_deg_s(bestIdx));
fprintf('Trim Quality (||dx||): %.4f\n', dx_norm(bestIdx));

fprintf('\nState Derivatives at Best Trim Point:\n');
state_names = {'dV/dt', 'dAlpha/dt', 'dGamma/dt', 'dq/dt', 'dh/dt'};
for k = 1:5
    fprintf('%-12s : %.6f\n', state_names{k}, dx_all(k, bestIdx));
end

%% Summary table
T = array2table(summary_table, ...
    'VariableNames', {'Velocity_knots', 'Throttle_hp', 'AoA_deg', ...
                      'PitchRate_deg_s', 'Elevator_deg', 'TrimError'});
disp(' ');
disp('=== Summary Table for All Trim Conditions ===');
disp(T);

%% ============================== FUNCTIONS ==============================
function J = objective_velocity_schedule_cost(x, modelName, velocity_settings, target_h, w_dx, w_h)
    [xi, ~, ~] = normalize_velocity(velocity_settings);
    elev = x(1) + x(2)*xi + x(3)*xi.^2;   % radians
    J = 0;
    for i = 1:numel(velocity_settings)
```

```
        V  = velocity_settings(i);
        el = elev(i);
        [~, u_trim, y_trim, dx_trim] = run_trim_velocity(modelName, V, el, target_h);
        dxn  = norm(dx_trim);
        h_err = (y_trim(5) - target_h);
        J = J + w_dx*(dxn^2) + w_h*(0.001*h_err)^2;
    end
end

function [x_trim, u_trim, y_trim, dx_trim] = run_trim_velocity(modelName, V_knots, elev_rad, target_h)
    % Trim with velocity & height fixed; elevator fixed by GA; throttle free.
    x0 = [V_knots; 0; 0; 0; target_h];   % [v; alpha; gamma; q; h]
    u0 = [elev_rad; 150];                % [elevator; throttle]
    y0 = [V_knots; 5; 1; 0; target_h];   % fix v, h as outputs

    ix = [];         % states free
    iu = [1];        % fix elevator only
    iy = [1, 5];     % fix velocity and height

    [x_trim, u_trim, y_trim, dx_trim] = trim(modelName, x0, u0, y0, ix, iu, iy);
end

function [xi, Vc, Vscale] = normalize_velocity(V)
    Vc     = mean([V(1), V(end)]);   % center
    Vscale = (V(end) - V(1))/2;      % half-range
    xi     = (V - Vc)/Vscale;
end

function idx = tournament_pick(F, k)
    n = numel(F);
    cand = randi(n, [k,1]);
    [~,m] = min(F(cand));
    idx = cand(m);
end
```

GA_TRIM_THROT.m

```
%% ================= GA for Trim (no toolboxes) ============================
clear; clc; close all; rng(1);

% --- Select model (same as your code) ---
[filename, pathname] = uigetfile('*.slx', 'Select your Simulink model file');
if isequal(filename,0), error('User canceled file selection.'); end
modelName = erase(filename, '.slx');

% --- Throttle sweep (hp) ---
throttle_hp = 70:2:110;
target_h = 5000;          % ft (your trim target)
w_dx = 1.0; w_h = 0.0;    % weights in cost

% --- Decision variables: elevator schedule coeffs (radians)  x=[a b c]
deg = pi/180;
nvar = 3;
lb = [-12*deg, -20*deg, -20*deg];
ub = [ +4*deg, +20*deg, +20*deg];

% --- Simple-GA settings (edit to taste) ---
popSize   = 50;
generations = 60;
eliteFrac = 0.1;          % top % copied each gen
pcross    = 0.8;          % crossover probability
pmut      = 0.2;          % mutation probability
mutSigma  = [2 5 5]*deg;  % mutation std dev per gene (radians)

% --- Parallel detection (optional) ---
usePar = false;
try
    if license('test','Distrib_Computing_Toolbox')
        p = gcp('nocreate'); if isempty(p), parpool; end
        usePar = true;
```

```
        fprintf('Parallel enabled.\n');
    end
catch, usePar = false; end

% Objective handle
obj = @(x) objective_schedule_cost(x, modelName, throttle_hp, target_h, w_dx, w_h);

% ---------------------- Initialize population ----------------------------
Pop = repmat(lb, popSize,1) + rand(popSize,nvar).*repmat((ub-lb),popSize,1);
BestHist = nan(generations,1);

% ---------------------- GA main loop ---------------------------------------
for g = 1:generations
    % Evaluate fitness (lower is better)
    F = nan(popSize,1);
    if usePar
        parfor i = 1:popSize
            F(i) = obj(Pop(i,:));
        end
    else
        for i = 1:popSize
            F(i) = obj(Pop(i,:));
        end
    end

    % Sort by fitness
    [F, idx] = sort(F);
    Pop = Pop(idx,:);
    BestHist(g) = F(1);

    fprintf('Gen %3d: best f = %.6g  | x(deg)=[%.2f %.2f %.2f]\n', ...
        g, F(1), Pop(1,:)*180/pi);

    % Elitism
    nelite = max(1, round(eliteFrac*popSize));
    NewPop = Pop(1:nelite,:);

    % Create offspring until population is refilled
    while size(NewPop,1) < popSize
        % Tournament selection (size 3)
        p1 = tournament_pick(F,3); p2 = tournament_pick(F,3);
        parent1 = Pop(p1,:); parent2 = Pop(p2,:);

        % Crossover
        if rand < pcross
            alpha = rand(1,nvar);
            child1 = alpha.*parent1 + (1-alpha).*parent2;
            child2 = alpha.*parent2 + (1-alpha).*parent1;
        else
            child1 = parent1; child2 = parent2;
        end

        % Mutation (Gaussian)
        child1 = child1 + (randn(1,nvar).*mutSigma).*(rand(1,nvar) < pmut);
        child2 = child2 + (randn(1,nvar).*mutSigma).*(rand(1,nvar) < pmut);

        % Bounds
        child1 = min(max(child1,lb),ub);
        child2 = min(max(child2,lb),ub);

        NewPop = [NewPop; child1; child2]; %#ok<AGROW>
    end

    % Keep exactly popSize
    Pop = NewPop(1:popSize,:);
end
```

```
% Final evaluation & best
F = nan(popSize,1);
for i = 1:popSize, F(i) = obj(Pop(i,:)); end
[bestF, bidx] = min(F); xbest = Pop(bidx,:);

fprintf('\n== Final best f = %.6g | coeffs (deg) = [%.2f %.2f %.2f]\n', ...
    bestF, xbest*180/pi);

% ---------- Inspect schedule and trim results ----------
xi = (throttle_hp - 90)/20;
elev_sched = xbest(1) + xbest(2)*xi + xbest(3)*xi.^2;   % radians

n = numel(throttle_hp);
alpha_arr = zeros(1,n); vel_arr = zeros(1,n); dx_arr = zeros(1,n); h_arr = zeros(1,n);

for i = 1:n
    hp = throttle_hp(i);
    [~, ~, y_trim, dx_trim, x_trim] = run_single_trim(modelName, hp, elev_sched(i), target_h);
    vel_arr(i)  = y_trim(1);
    h_arr(i)    = y_trim(5);
    alpha_arr(i)= x_trim(2);
    dx_arr(i)   = norm(dx_trim);
end

figure; plot(1:generations, BestHist, 'LineWidth',2);
xlabel('Generation'); ylabel('Best objective'); grid on; title('GA convergence');

figure; plot(throttle_hp, elev_sched*180/pi, '-o','LineWidth',2);
xlabel('Throttle (hp)'); ylabel('Elevator (deg)'); grid on; title('Optimized elevator schedule');

figure;
yyaxis left; plot(throttle_hp, dx_arr, '-o','LineWidth',2); ylabel('||dx||');
yyaxis right; plot(throttle_hp, alpha_arr*180/pi, '-s','LineWidth',2); ylabel('\alpha (deg)');
xlabel('Throttle (hp)'); grid on; title('Trim residual & AoA');

[dx_min, kbest] = min(dx_arr);
fprintf('Best at %.1f hp: ||dx||=%.3g, elev=%.2f deg, AoA=%.2f deg, V=%.1f kt, h=%.0f ft\n', ...
    throttle_hp(kbest), dx_min, elev_sched(kbest)*180/pi, alpha_arr(kbest)*180/pi, vel_arr(kbest), h_arr(kbest));

%% ========================= Helpers ======================================
function J = objective_schedule_cost(x, modelName, throttle_hp, target_h, w_dx, w_h)
    xi = (throttle_hp - 90)/20;
    elev = x(1) + x(2)*xi + x(3)*xi.^2;   % radians
    J = 0;
    for k = 1:numel(throttle_hp)
        hp = throttle_hp(k);
        [~, ~, y_trim, dx_trim] = run_single_trim(modelName, hp, elev(k), target_h);
        J = J + w_dx*(norm(dx_trim)^2) + w_h*(0.001*(y_trim(5)-target_h))^2;
    end
end

function [x_trim, u_trim, y_trim, dx_trim, x_out] = run_single_trim(modelName, throttle_hp, elev_rad, target_h)
    x0 = [120; 0; 0; 0; target_h];
    u0 = [elev_rad; throttle_hp];     % fix both inputs
    y0 = [120; 5; 1; 0; target_h];
    ix = []; iu = [1 2]; iy = [5];
    [x_trim, u_trim, y_trim, dx_trim] = trim(modelName, x0, u0, y0, ix, iu, iy);
    x_out = x_trim;
end

function idx = tournament_pick(F, k)
    n = numel(F);
    cand = randi(n, [k,1]);
    [~,m] = min(F(cand));
    idx = cand(m);
```

```
end
```

GA_TRIM_altitude.m

```
%% === Altitude Sweep + GA throttle schedule (elevator free, guarded) ===
clear; clc; close all; rng(1);

% --- Select model ---
[filename, pathname] = uigetfile('*.slx', 'Select your Simulink model file');
if isequal(filename, 0), error('User canceled file selection.'); end
modelName = erase(filename, '.slx');

% --- Altitude sweep ---
altitudes = 5000:100:7000;     % ft
n = numel(altitudes);

% === GA decision: throttle schedule vs altitude ============================
% Quadratic schedule: T(h) = a + b*xi + c*xi^2, xi = normalized altitude
nvar = 3;
% Reasonable bounds for coefficients (hp); schedule is clamped again in objective
lb   = [  0, -150, -150];
ub   = [200,  150,  150];

% Physical throttle clamp (used inside objective)
T_MIN = 70;   % hp  (edit to your engine min)
T_MAX = 130;  % hp  (edit to your engine max)

% Objective weights
w_dx  = 1.0;  % weight on trim residuals
w_elevSoft = 0.0; % optional soft penalty on elevator extremes (deg), set >0 to enable

% GA meta settings
popSize     = 50;
generations = 60;

% ---------- Robust toolbox / parallel checks ----------
hasGAFcn   = ~isempty(which('ga'));
hasGADSVer = ~isempty(ver('gads'));
hasOptOpts = ~isempty(which('optimoptions'));
useGADS    = hasGAFcn && hasGADSVer && hasOptOpts && license('test','GADS_Toolbox');

usePCT = ~isempty(which('parpool')) && license('test','Distrib_Computing_Toolbox');
if usePCT
   try
      if isempty(gcp('nocreate')), parpool; end
      fprintf('Parallel enabled.\n');
   catch ME
      warning('Could not start parallel pool: %s. Continuing single-threaded.', ME.message);
      usePCT = false;
   end
end

% === Objective handle (guarded) ===
obj = @(x) objective_throttle_schedule_cost(x, modelName, altitudes, w_dx, w_elevSoft, T_MIN, T_MAX);

% ===================== Try built-in GA, else fallback =====================
BestHist = [];
if useGADS
   fprintf('Using built-in GA (Global Optimization Toolbox).\n');
   try
      opts = optimoptions('ga', ...
         'PopulationSize',       popSize, ...
```

```
            'MaxGenerations',       generations, ...
            'UseParallel',          usePCT, ...
            'Display',              'iter', ...
            'FunctionTolerance',    1e-6, ...
            'MaxStallGenerations',  20);
        [xbest, bestF] = ga(obj, nvar, [], [], [], [], lb, ub, [], opts);
        fprintf('\n== GA (toolbox) best f = %.6g | coeffs = [%.3f %.3f %.3f]\n', bestF, xbest);
    catch ME
        warning('Built-in GA failed (%s). Falling back to custom GA.', ME.message);
        useGADS = false;
    end
end

if ~useGADS
    fprintf('Using custom GA (no GADS / fallback).\n');
    eliteFrac = 0.1; pcross = 0.8; pmut = 0.2; mutSigma = [5 10 10]; % hp units
    usePar = usePCT;

    Pop = repmat(lb, popSize,1) + rand(popSize,nvar).*repmat((ub-lb),popSize,1);
    BestHist = nan(generations,1);

    for g = 1:generations
        F = nan(popSize,1);
        if usePar
            parfor i = 1:popSize, F(i) = obj(Pop(i,:)); end
        else
            for i = 1:popSize, F(i) = obj(Pop(i,:)); end
        end

        [F,idx] = sort(F); Pop = Pop(idx,:); BestHist(g) = F(1);
        fprintf('Gen %3d: best f=%.6g | coeffs=[%.2f %.2f %.2f]\n', g, F(1), Pop(1,:));

        nelite = max(1, round(eliteFrac*popSize));
        NewPop = Pop(1:nelite,:);

        while size(NewPop,1) < popSize
            p1 = tournament_pick(F,3); p2 = tournament_pick(F,3);
            pa = Pop(p1,:); pb = Pop(p2,:);
            if rand < pcross
                a = rand(1,nvar);
                c1 = a.*pa + (1-a).*pb;
                c2 = a.*pb + (1-a).*pa;
            else
                c1 = pa; c2 = pb;
            end
            c1 = c1 + (randn(1,nvar).*mutSigma).*(rand(1,nvar) < pmut);
            c2 = c2 + (randn(1,nvar).*mutSigma).*(rand(1,nvar) < pmut);
            c1 = min(max(c1, lb), ub);
            c2 = min(max(c2, lb), ub);
            NewPop = [NewPop; c1; c2]; %#ok<AGROW>
        end

        Pop = NewPop(1:popSize,:);
    end

    F = arrayfun(@(i) obj(Pop(i,:)), 1:popSize).';
    [bestF, b] = min(F); xbest = Pop(b,:);
    fprintf('\n== GA (custom) best f=%.6g | coeffs=[%.2f %.2f %.2f]\n', bestF, xbest);
end

% Build final throttle schedule (clamped)
xi = normalize_altitude(altitudes);
throttle_sched = xbest(1) + xbest(2)*xi + xbest(3)*xi.^2;
throttle_sched = min(max(throttle_sched, T_MIN), T_MAX);

% ================== Run sweep (throttle fixed, elevator free) ============
```

```
velocity_array   = zeros(1, n);
throttle_array   = zeros(1, n);
alpha_array      = zeros(1, n);
elevator_array   = zeros(1, n);
pitch_rate_array = zeros(1, n);
dx_norm          = zeros(1, n);
dx_all           = zeros(n, 5);

for i = 1:n
    h = altitudes(i);
    T = throttle_sched(i);

    % Initial guesses
    x0 = [120; 0.05; 0; 0; h];
    u0 = [0; T];                % elevator free, throttle fixed by schedule
    y0 = [120; 5; 1; 0; h];

    ix = []; iu = [2]; iy = [5]; % fix throttle (2), fix altitude
    [ok, x_trim, u_trim, y_trim, dx_trim] = safe_trim_alt(modelName, x0, u0, y0, ix, iu, iy);

    if ~ok
        velocity_array(i)   = NaN;
        throttle_array(i)   = T;
        alpha_array(i)      = NaN;
        elevator_array(i)   = NaN;
        pitch_rate_array(i) = NaN;
        dx_norm(i)          = Inf;
        dx_all(i,:)         = NaN(1,5);
    else
        velocity_array(i)   = y_trim(1);
        throttle_array(i)   = u_trim(2);
        alpha_array(i)      = x_trim(2);
        elevator_array(i)   = u_trim(1);
        pitch_rate_array(i) = x_trim(4);
        dx_norm(i)          = norm(dx_trim);
        dx_all(i,:)         = dx_trim(:)';
    end
end

% Convert to degrees
alpha_deg = rad2deg(alpha_array);
elev_deg  = rad2deg(elevator_array);
q_deg_s   = rad2deg(pitch_rate_array);

% ============================== Plots ====================================
if ~isempty(BestHist)
    figure; plot(1:numel(BestHist), BestHist, 'LineWidth',2);
    xlabel('Generation'); ylabel('Best objective'); grid on;
    title('GA convergence (custom)');
end

figure;
yyaxis left
plot(altitudes, velocity_array, '-o', 'LineWidth', 2, 'Color', '#0072BD'); ylabel('Velocity (ft/s)');
yyaxis right
plot(altitudes, throttle_array, '-s', 'LineWidth', 2, 'Color', '#D95319'); ylabel('Throttle (hp)');
xlabel('Altitude (ft)'); title('Trimmed Velocity and Throttle vs Altitude');
legend('Velocity','Throttle','Location','best'); grid on;

figure;
plot(altitudes, alpha_deg, '-o', 'LineWidth', 2); hold on;
plot(altitudes, elev_deg,  '-s', 'LineWidth', 2);
plot(altitudes, q_deg_s,   '-^', 'LineWidth', 2);
xlabel('Altitude (ft)'); ylabel('Angle / Rate (deg / deg/s)');
title('AoA, Elevator Deflection (free), and Pitch Rate vs Altitude');
legend('Angle of Attack','Elevator (free)','Pitch Rate'); grid on;
```

```
figure;
plot(altitudes, throttle_sched, '-d', 'LineWidth', 2);
xlabel('Altitude (ft)'); ylabel('Throttle Schedule (hp)');
title('GA-Fitted Throttle Schedule vs Altitude'); grid on;

% ============================ Best trim report ==============================
[~, bestIdx] = min(dx_norm);
fprintf('\n=== Best Trim Condition (Fixed Altitude, Elevator Free) ===\n');
fprintf('Altitude          : %.0f ft\n', altitudes(bestIdx));
fprintf('Velocity          : %.2f ft/s\n', velocity_array(bestIdx));
fprintf('Throttle (sched)  : %.2f hp\n', throttle_array(bestIdx));
fprintf('Elevator Deflection: %.2f deg\n', elev_deg(bestIdx));
fprintf('Angle of Attack    : %.2f deg\n', alpha_deg(bestIdx));
fprintf('Pitch Rate (q)     : %.2f deg/s\n', q_deg_s(bestIdx));
fprintf('Trim Quality (||dx||): %.6f\n', dx_norm(bestIdx));

fprintf('\n=== State Derivatives at Best Trim Point ===\n');
state_names = {'dV/dt','dAlpha/dt','dGamma/dt','dq/dt','dh/dt'};
for k = 1:5
   fprintf('%-12s : %.6e\n', state_names{k}, dx_all(bestIdx, k));
end

%% ============================== FUNCTIONS ================================

function J = objective_throttle_schedule_cost(x, modelName, altitudes, w_dx, w_elevSoft, T_MIN, T_MAX)
   % Build & clamp throttle schedule
   xi = normalize_altitude(altitudes);
   T  = x(1) + x(2)*xi + x(3)*xi.^2;
   T  = min(max(T, T_MIN), T_MAX);

   J = 0;
   for i = 1:numel(altitudes)
      h = altitudes(i); Ti = T(i);
      [ok, x_trim, u_trim, y_trim, dx_trim] = safe_trim_alt(modelName, ...
         [120; 0.05; 0; 0; h], [0; Ti], [120; 5; 1; 0; h], [], [2], [5]);

      if ~ok || any(~isfinite(dx_trim))
         J = J + 1e6;                  % heavy penalty on failures
      else
         J = J + w_dx * (norm(dx_trim)^2);
         if w_elevSoft > 0
            elev_deg = abs(rad2deg(u_trim(1)));
            J = J + w_elevSoft * max(0, elev_deg - 15)^2;  % soft keep |elev|<=15°
         end
      end
   end
end

function [ok, x_trim, u_trim, y_trim, dx_trim] = safe_trim_alt(modelName, x0, u0, y0, ix, iu, iy)
   % Robust trim wrapper: catches errors and invalid numbers
   ok = true; x_trim=[]; u_trim=[]; y_trim=[]; dx_trim=nan(5,1);

   % Ensure doubles & column vectors
   x0 = double(x0(:)); u0 = double(u0(:)); y0 = double(y0(:));
   try
      [x_trim, u_trim, y_trim, dx_trim] = trim(modelName, x0, u0, y0, ix, iu, iy);

      bad = any(~isfinite([x_trim; u_trim; y_trim; dx_trim]));
      if bad, ok=false; end
   catch
      ok = false;
   end
end

function xi = normalize_altitude(h_ft)
```

```
    hc     = mean([h_ft(1), h_ft(end)]);
    hscale = (h_ft(end) - h_ft(1))/2;
    xi     = (h_ft - hc)/max(hscale, eps);
end

function idx = tournament_pick(F, k)
    n = numel(F);
    cand = randi(n, [k,1]);
    [~,m] = min(F(cand));
    idx = cand(m);
end
```

Trim_run_velocity.m

```
clear all; clc;

% Prompt user to select the Simulink model file
[filename, pathname] = uigetfile('*.slx', 'Select your Simulink model file');
if isequal(filename, 0)
    error('User canceled file selection.');
end

modelName = erase(filename, '.slx');

% Velocity sweep (in knots) from 110 to 160 in steps of 5
velocity_settings = 110:5:160;
n = length(velocity_settings);

% Preallocate arrays
alpha_array     = zeros(1, n);      % angle of attack
q_array         = zeros(1, n);      % pitch rate
elevator_array  = zeros(1, n);      % elevator input
throttle_array  = zeros(1, n);      % resulting throttle
dx_norm         = zeros(1, n);      % trim quality
dx_all          = zeros(5, n);      % all state derivatives for review

% Table data collection
summary_table = [];

for i = 1:n
    velocity = velocity_settings(i);

    % Initial guesses
    x0 = [velocity; 0; 0; 0; 5000];  % v, alpha, gamma, q, height
    u0 = [0; 150];                   % elevator, throttle guess
    y0 = [velocity; 5; 1; 0; 5000];  % desired outputs

    % Fix velocity and height
    ix = [];
    iu = [];          % Allow throttle to vary
    iy = [1, 5];      % Fix output: velocity and height

    % Run trim
    [x_trim, u_trim, y_trim, dx_trim] = trim(modelName, x0, u0, y0, ix, iu, iy);

    % Store results
    alpha_array(i)    = x_trim(2);
    q_array(i)        = x_trim(4);
    elevator_array(i) = u_trim(1);
    throttle_array(i) = u_trim(2);
    dx_norm(i)        = norm(dx_trim);
    dx_all(:, i)      = dx_trim;  % Store full state derivative

    % Store in table format
```

```
    summary_table = [summary_table; ...
        velocity, u_trim(2), rad2deg(x_trim(2)), rad2deg(x_trim(4)), rad2deg(u_trim(1)), norm(dx_trim)];
end

% Convert radians to degrees
alpha_deg = rad2deg(alpha_array);
q_deg_s   = rad2deg(q_array);
elev_deg  = rad2deg(elevator_array);

%% Plot 1: Combined Plot of Throttle and AoA vs Velocity
figure;
yyaxis left
plot(velocity_settings, throttle_array, '-o', 'LineWidth', 2, 'Color', '#D95319');
ylabel('Required Throttle (hp)');

yyaxis right
plot(velocity_settings, alpha_deg, '-s', 'LineWidth', 2, 'Color', '#0072BD');
ylabel('Angle of Attack (deg)');

xlabel('Trim Velocity (knots)');
title('Required Throttle and AoA vs Velocity');
legend('Throttle', 'Angle of Attack', 'Location', 'best');
grid on;

%% Plot 2: Separate Plot - Throttle vs Velocity
figure;
plot(velocity_settings, throttle_array, '-o', 'LineWidth', 2, 'Color', '#EDB120');
grid on;
xlabel('Trim Velocity (knots)');
ylabel('Required Throttle (hp)');
title('Throttle Requirement vs Velocity');
legend('Throttle vs Velocity', 'Location', 'best');

%% Display best trim point (closest to zero derivatives)
[~, bestIdx] = min(dx_norm);
fprintf('\n=== Best Trim Condition ===\n');
fprintf('Velocity          : %.1f knots\n', velocity_settings(bestIdx));
fprintf('Required Throttle  : %.2f hp\n', throttle_array(bestIdx));
fprintf('Angle of Attack    : %.2f deg\n', alpha_deg(bestIdx));
fprintf('Elevator Deflection: %.2f deg\n', elev_deg(bestIdx));
fprintf('Pitch Rate (q)     : %.2f deg/s\n', q_deg_s(bestIdx));
fprintf('Trim Quality (||dx||): %.4f\n', dx_norm(bestIdx));

% Display individual state derivatives
fprintf('\nState Derivatives at Best Trim Point:\n');
state_names = {'dV/dt', 'dAlpha/dt', 'dGamma/dt', 'dq/dt', 'dh/dt'};
for k = 1:5
    fprintf('%-12s : %.6f\n', state_names{k}, dx_all(k, bestIdx));
end

%% Display summary table
T = array2table(summary_table, ...
    'VariableNames', {'Velocity_knots', 'Throttle_hp', 'AoA_deg', ...
                      'PitchRate_deg_s', 'Elevator_deg', 'TrimError'});
disp(' ');
disp('=== Summary Table for All Trim Conditions ===');
disp(T);
```

Trim_run_throttle.m

```
clear all; clc;
```

```
% Prompt user to select the Simulink model file
[filename, pathname] = uigetfile('*.slx', 'Select your Simulink model file');
if isequal(filename, 0)
   error('User canceled file selection.');
end

modelName = erase(filename, '.slx');

% Throttle sweep from 70 to 110 hp in increments of 2
throttle_settings = 70:2:110;
n = length(throttle_settings);

% Preallocate arrays
velocity_array   = zeros(1, n);   % Trimmed velocity (knots)
alpha_array      = zeros(1, n);   % Angle of attack (rad)
elevator_array   = zeros(1, n);   % Elevator input (rad)
dx_norm          = zeros(1, n);   % Trim quality

% Loop through throttle settings
for i = 1:n
   throttle = throttle_settings(i);

   % Initial guess
   x0 = [120; 0; 0; 0; 5000];     % v, alpha, gamma, q, height
   u0 = [0; throttle];           % elevator, throttle (fixed)
   y0 = [120; 5; 1; 0; 5000];    % desired outputs (let velocity float)

   ix = [];          % Let all states vary
   iu = [2];         % Fix throttle
   iy = [5];         % Only fix height (let velocity float)

   % Run trim
   [x_trim, u_trim, y_trim, dx_trim] = trim(modelName, x0, u0, y0, ix, iu, iy);

   % Store results
   velocity_array(i)  = y_trim(1);       % Trimmed velocity (knots)
   alpha_array(i)     = x_trim(2);       % Angle of attack (rad)
   elevator_array(i)  = u_trim(1);       % Elevator deflection (rad)
   dx_norm(i)         = norm(dx_trim);   % Trim quality
end

% Convert radians to degrees for plotting
alpha_deg = rad2deg(alpha_array);
elev_deg  = rad2deg(elevator_array);

%% === Plot 1: Velocity vs Throttle ===
figure;
plot(throttle_settings, velocity_array, '-o', 'LineWidth', 2, 'Color', '#0072BD');
xlabel('Throttle (hp)');
ylabel('Trimmed Velocity (knots)');
title('Trimmed Velocity vs Throttle');
grid on;
legend('Velocity vs Throttle', 'Location', 'best');

%% === Plot 2: Elevator Deflection vs Throttle ===
figure;
plot(throttle_settings, elev_deg, '-s', 'LineWidth', 2, 'Color', '#D95319');
xlabel('Throttle (hp)');
ylabel('Elevator Deflection (deg)');
title('Elevator Deflection vs Throttle');
grid on;
legend('Elevator vs Throttle', 'Location', 'best');

%% Display best trim condition
[~, bestIdx] = min(dx_norm);
fprintf('\n=== Best Trim Condition ===\n');
```

```
fprintf('Throttle          : %.2f hp\n', throttle_settings(bestIdx));
fprintf('Trimmed Velocity  : %.2f knots\n', velocity_array(bestIdx));
fprintf('Angle of Attack   : %.4f rad\n', alpha_array(bestIdx));
fprintf('Elevator Deflection: %.4f rad\n', elevator_array(bestIdx));
fprintf('Trim Quality (||dx||): %.6f\n', dx_norm(bestIdx));
```

Trim_run_altitude.m

```
clear all; clc;

% Prompt user to select the Simulink model file
[filename, pathname] = uigetfile('*.slx', 'Select your Simulink model file');
if isequal(filename, 0)
   error('User canceled file selection.');
end

modelName = erase(filename, '.slx');

% Altitude sweep from 5000 to 7000 ft in increments of 100
altitudes = 5000:100:7000;
n = length(altitudes);

% Preallocate arrays
velocity_array   = zeros(1, n);
throttle_array   = zeros(1, n);
alpha_array      = zeros(1, n);
elevator_array   = zeros(1, n);
pitch_rate_array = zeros(1, n);
dx_norm          = zeros(1, n);
dx_all           = zeros(n, 5);  % To store all dx values

% Loop through each altitude
for i = 1:n
   h = altitudes(i);

   % Initial guesses
   x0 = [120; 0.05; 0; 0; h];    % v, alpha, gamma, q, height
   u0 = [0; 75];                 % elevator, throttle
   y0 = [120; 5; 1; 0; h];       % desired outputs

   ix = [];
   iu = [];
   iy = [5];                     % Fix only altitude

   % Run trim
   [x_trim, u_trim, y_trim, dx_trim] = trim(modelName, x0, u0, y0, ix, iu, iy);

   % Store outputs
   velocity_array(i)   = y_trim(1);
   throttle_array(i)   = u_trim(2);
   alpha_array(i)      = x_trim(2);
   elevator_array(i)   = u_trim(1);
   pitch_rate_array(i) = x_trim(4);
   dx_norm(i)          = norm(dx_trim);
   dx_all(i, :)        = dx_trim(:)';  % store derivative vector
end

% Convert to degrees
alpha_deg = rad2deg(alpha_array);
elev_deg  = rad2deg(elevator_array);
q_deg_s   = rad2deg(pitch_rate_array);

%% Plot 1: Velocity and Throttle vs Altitude
figure;
```

```
yyaxis left
plot(altitudes, velocity_array, '-o', 'LineWidth', 2, 'Color', '#0072BD');
ylabel('Velocity (ft/s)');
yyaxis right
plot(altitudes, throttle_array, '-s', 'LineWidth', 2, 'Color', '#D95319');
ylabel('Throttle (hp)');
xlabel('Altitude (ft)');
title('Trimmed Velocity and Throttle vs Altitude');
legend('Velocity', 'Throttle', 'Location', 'best');
grid on;

%% Plot 2: AoA, Elevator, and Pitch Rate vs Altitude
figure;
plot(altitudes, alpha_deg, '-o', 'LineWidth', 2); hold on;
plot(altitudes, elev_deg, '-s', 'LineWidth', 2);
plot(altitudes, q_deg_s, '-^', 'LineWidth', 2);
xlabel('Altitude (ft)');
ylabel('Angle / Rate (deg / deg/s)');
title('AoA, Elevator Deflection, and Pitch Rate vs Altitude');
legend('Angle of Attack', 'Elevator Deflection', 'Pitch Rate');
grid on;

%% Display Best Trim Condition and Derivatives
[~, bestIdx] = min(dx_norm);

fprintf('\n=== Best Trim Condition (Free Trim at Fixed Altitude) ===\n');
fprintf('Altitude          : %.0f ft\n', altitudes(bestIdx));
fprintf('Velocity          : %.2f ft/s\n', velocity_array(bestIdx));
fprintf('Throttle          : %.2f hp\n', throttle_array(bestIdx));
fprintf('Angle of Attack   : %.4f rad\n', alpha_array(bestIdx));
fprintf('Elevator Deflection: %.4f rad\n', elevator_array(bestIdx));
fprintf('Pitch Rate (q)    : %.4f rad/s\n', pitch_rate_array(bestIdx));
fprintf('Trim Quality (||dx||): %.6f\n', dx_norm(bestIdx));

% Print state derivatives at best trim
fprintf('\n=== State Derivatives at Best Trim Point ===\n');
fprintf('dV/dt     : %.6e\n', dx_all(bestIdx, 1));
fprintf('dAlpha/dt : %.6e\n', dx_all(bestIdx, 2));
fprintf('dGamma/dt : %.6e\n', dx_all(bestIdx, 3));
fprintf('dq/dt     : %.6e\n', dx_all(bestIdx, 4));
fprintf('dh/dt     : %.6e\n', dx_all(bestIdx, 5));
```

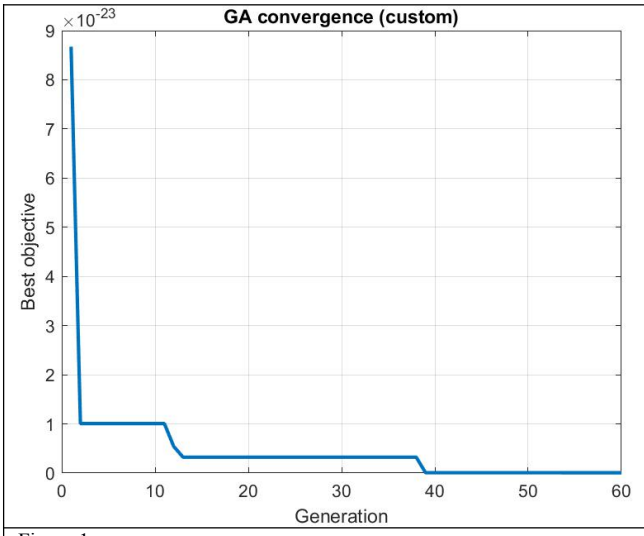


Figure 1

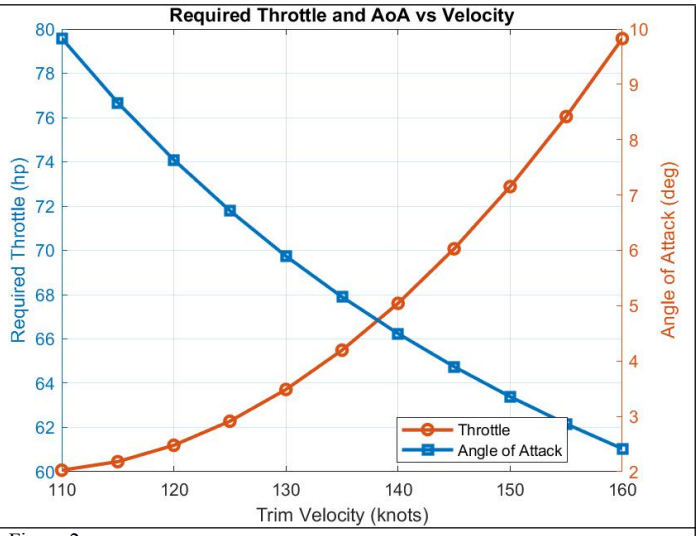


Figure 2

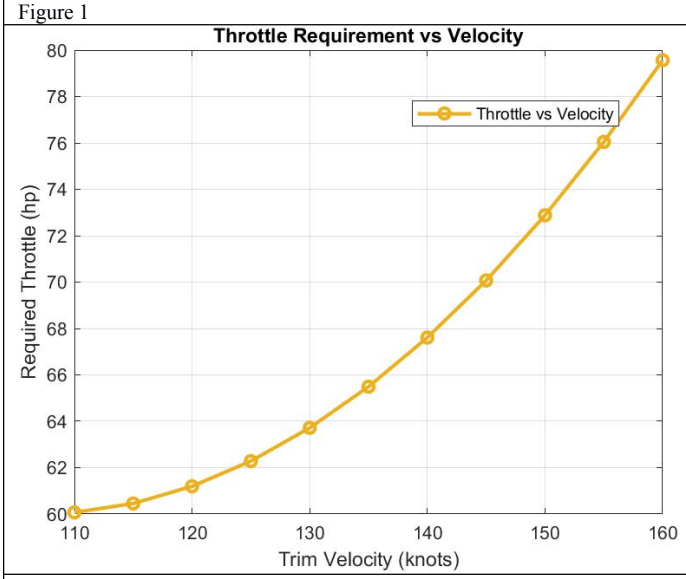


Figure 3

=== Best Trim Condition ===
Velocity : 135.0 knots
Required Throttle : 65.49 hp
Angle of Attack : 5.16 deg
Elevator Deflection: -0.78 deg
Pitch Rate (q) : 0.00 deg/s
Trim Quality (||dx||): 0.0000

State Derivatives at Best Trim Point:
dV/dt : 0.000000
dAlpha/dt : -0.000000
dGamma/dt : 0.000000
dq/dt : 0.000000
dh/dt : 0.000000

=== Summary Table for All Trim Conditions ===

Velocity_knots	Throttle_hp	AoA_deg	PitchRate_deg_s	Elevator_deg	TrimError
110	60.069	9.8323	-8.8161e-22	-3.3292	2.2225e-13
115	60.455	8.6685	-5.6799e-22	-2.6934	8.4217e-14
120	61.195	7.6363	1.861e-21	-2.1294	2.2371e-14
125	62.282	6.7172	1.7147e-21	-1.6273	5.5406e-14
130	63.714	5.8959	-1.042e-21	-1.1786	9.1911e-15
135	65.489	5.1593	6.9314e-24	-0.77612	1.446e-16
140	67.607	4.4964	2.8514e-22	-0.41398	3.1492e-14
145	70.07	3.898	1.6829e-21	-0.08705	2.6394e-14
150	72.881	3.3562	-7.8288e-22	0.20898	2.0299e-15
155	76.045	2.8641	-8.4104e-25	0.47783	6.9504e-16
160	79.565	2.416	-3.9925e-22	0.722	

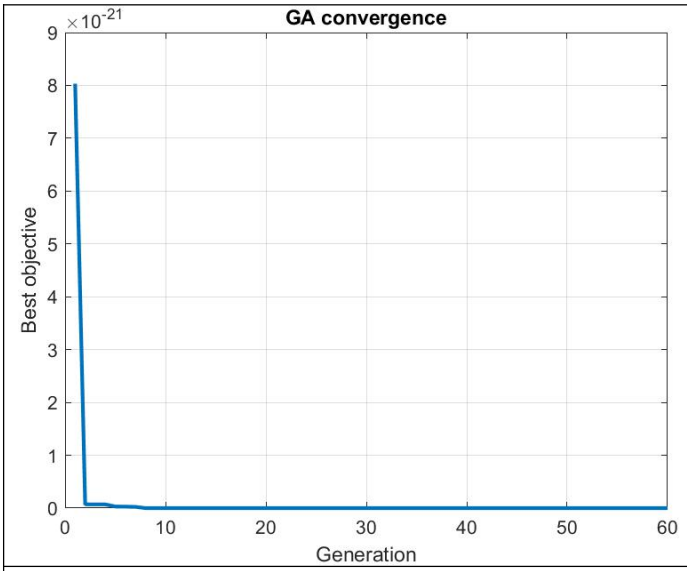


Figure 1

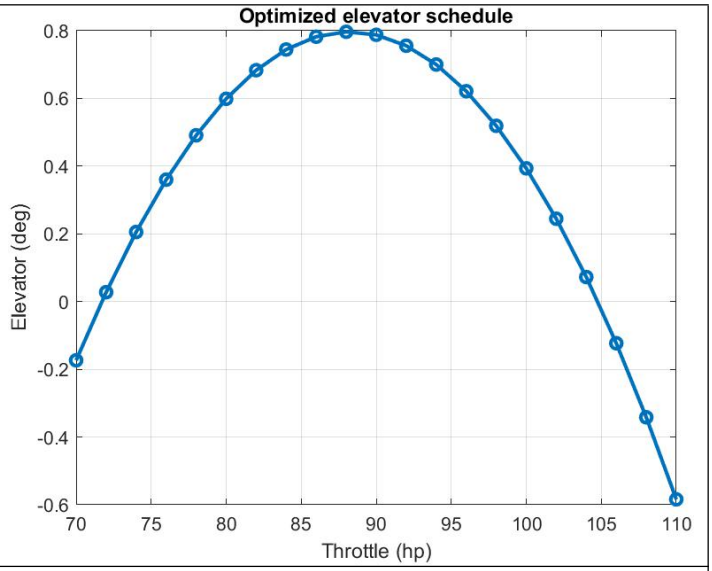


Figure 2

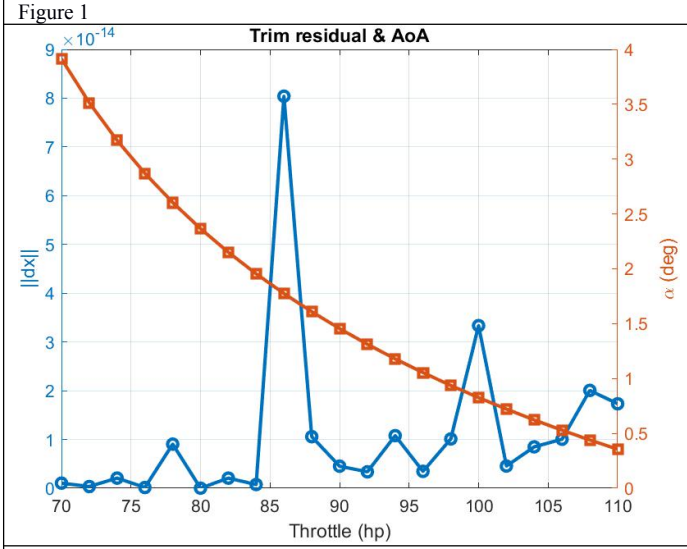


Figure 3

Best at 80.0 hp: ||dx||=1.12e-17, elev=0.60 deg, AoA=2.37 deg, V=160.7 kt, h=5000 ft

GA_TRIM_velocity.m Output

GA_TRIM_THROT.m Output

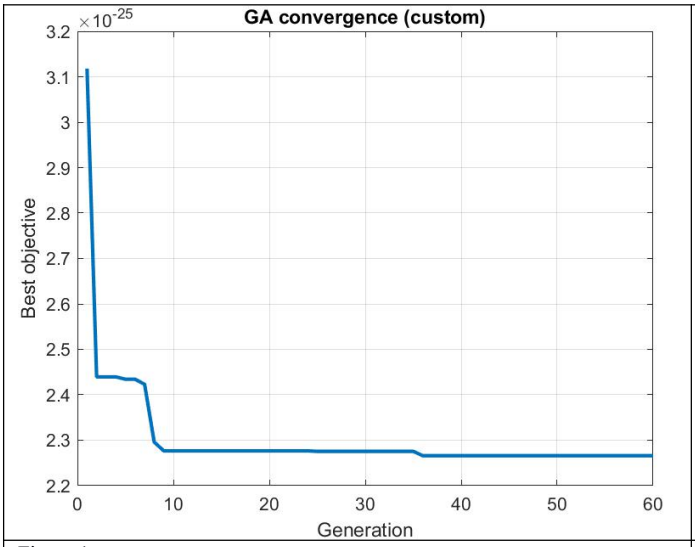


Figure 1

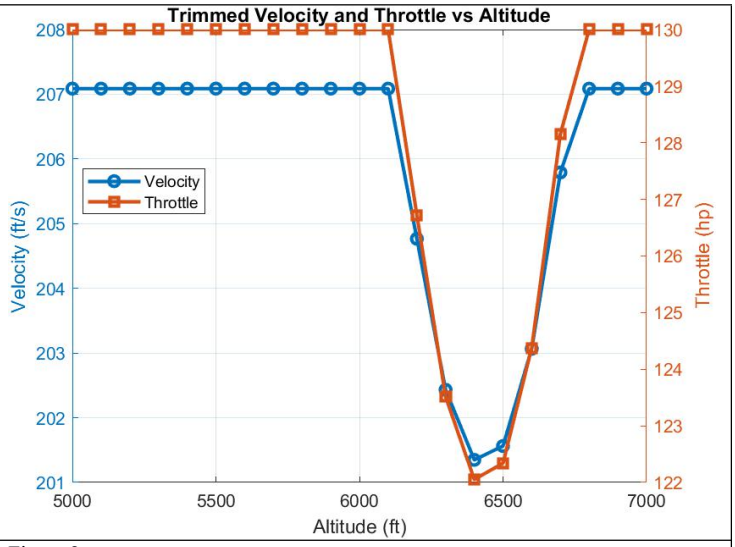


Figure 2

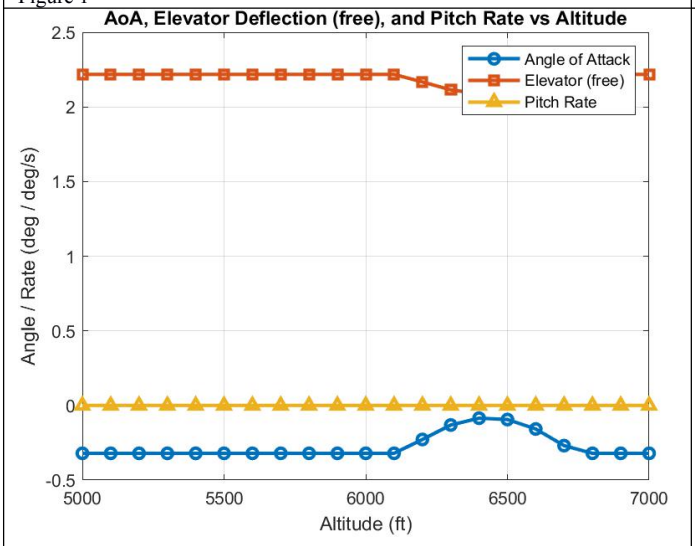
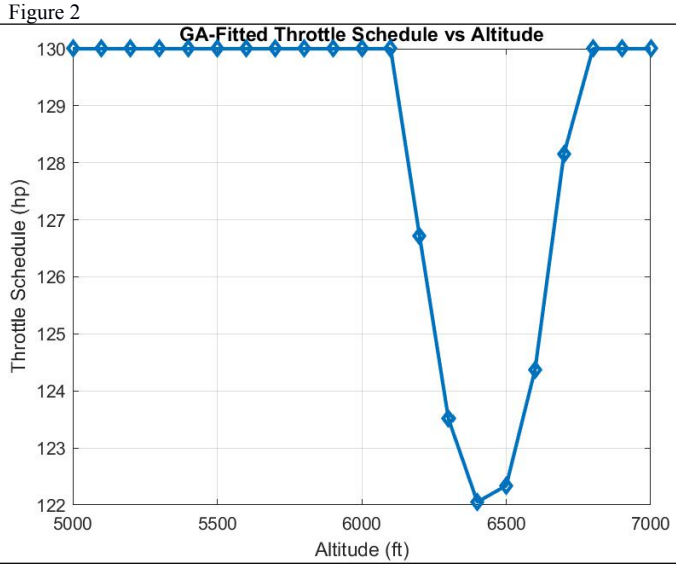


Figure 3




```
=== Best Trim Condition (Fixed Altitude, Elevator Free) ===
Altitude          : 6300 ft
Velocity          : 202.43 ft/s
Throttle (sched)  : 123.51 hp
Elevator Deflection: 2.11 deg
Angle of Attack   : -0.13 deg
Pitch Rate (q)    : -0.00 deg/s
Trim Quality (||dx||): 0.000000

=== State Derivatives at Best Trim Point ===
dV/dt       : 0.000000e+00
dAlpha/dt   : 1.430931e-16
dGamma/dt   : -5.693948e-26
dq/dt       : -3.884170e-16
dh/dt       : 0.000000e+00
```

GA_TRIM_altitude.mOutput